

Serverless Web Application using Lambda + API Gateway

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Anil Kumar Varanasi, APSSDC

Submitted by

LINGALA SARACHANDRA, 23BQ1A05C2, VVIT GUNTUR, (Batch-1)

JAGADALA NAGENDRA BABU, 23BQ1A0591, VVIT GUNTUR, (Batch-1)

SUMALA JAGADEESH, 23BQ1A0592, VVIT GUNTUR, (Batch-1)

PACHIPALA NARESH KUMAR, 23BQ1A05G3, VVIT GUNTUR, (Batch-1)

GULLAPUDI DURGA PRASAD, 23BQ1A0586, VVIT GUNTUR, (Batch-1)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr Anil Kumar Varanasi, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

LINGALA SARACHANDRA,7013574710, lingalasarachandra@gmail.com, 23bq1a05c2.
JAGADALA NAGENDRA BABU,9951068775,nanijagadala@gmail.com,23BQ1A0591.
PACHIPALA NARESH KUMAR,9347930314,p.nareshkumar9347@gmail.com,23BQ1A05G3.
GULLAPUDI DURGA PRASAD, 6301749299 ,durgaprasadg299@gmail.com, 23bq1a0586.
JAGADEESH SUMALA ,8341767224,karthiksumala1@gmail.com,23BQ1A0592

ABSTRACT

presents the implementation of **QuickShare**, a Serverless Web Application developed using **Amazon Web Services (AWS)** to provide a scalable, secure, and cost-effective platform for storing and retrieving information without managing traditional server infrastructure. The primary objective of the project is to demonstrate the practical implementation of serverless computing by utilizing AWS managed services to build a highly available, event-driven web application with minimal operational overhead.

The application consists of a responsive frontend developed using **REACT**, and **JavaScript**, which communicates with a serverless backend through **Amazon API Gateway**. Whenever a user performs an action such as creating, accessing, or retrieving shared content, the request is securely forwarded to API Gateway, which automatically invokes an **AWS Lambda** function. The Lambda function executes the required business logic, processes user requests, interacts with a **MongoDB** database for storing and retrieving application data, and returns a structured JSON response to the frontend, ensuring seamless communication between the user interface and backend services.

The architecture follows a completely serverless model by utilizing **AWS Lambda** for backend computation, **Amazon API Gateway** for REST API management, **MongoDB** as the cloud database for persistent data storage, **Amazon S3** for hosting static frontend files (where applicable), **AWS Identity and Access Management (IAM)** for secure access control, and **Amazon CloudWatch** for application monitoring and logging. These managed services work together to provide a reliable, fault-tolerant, and highly available cloud-native application without requiring the provisioning or maintenance of traditional application servers.

This solution offers several advantages, including automatic scaling, efficient resource utilization, reduced infrastructure costs, and simplified application management. Since AWS

automatically provisions computing resources based on incoming requests, developers can focus on implementing application functionality rather than managing servers, operating systems, or hardware infrastructure. The serverless architecture enables the application to efficiently handle varying workloads while maintaining consistent performance, reliability, and security.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION	6
2.	METHODOLOGY.....	7
3.	SYSTEM DESIGN / ARCHITECTURE	10
4.	IMPLEMENTATION	19
5.	RESULTS	25
6.	CONCLUSION.....	28

1. INTRODUCTION

Cloud computing has transformed modern application development by providing scalable, reliable, and cost-effective computing resources over the internet. Among its various service models, **serverless computing** has become a popular approach because it allows developers to build and deploy applications without managing servers or underlying infrastructure. In this model, computing resources are automatically allocated by the cloud provider whenever a request is received, reducing operational complexity and infrastructure costs.

The **QuickShare** project is a serverless web application developed using **Amazon Web Services (AWS)**. The application enables users to securely store and retrieve shared information through a simple web interface. The frontend is built using **REACT**, and **JavaScript**, while the backend is implemented using **AWS Lambda** and exposed through **Amazon API Gateway**. User data is stored in **MongoDB**, providing flexible and efficient NoSQL database support.

The application follows an **event-driven architecture**, where API Gateway receives HTTP requests from users and invokes AWS Lambda functions to execute the required business logic. The processed data is then stored or retrieved from MongoDB, and the response is returned to the frontend in JSON format. This architecture provides automatic scalability, high availability, and efficient resource utilization without requiring dedicated application servers.

The project also uses **AWS Identity and Access Management (IAM)** to provide secure access between AWS services and **Amazon CloudWatch** to monitor application logs and performance. Overall, the QuickShare project demonstrates how serverless technologies can be used to develop modern, secure, scalable, and cost-effective cloud-based web applications with minimal infrastructure management.

2. METHODOLOGY

The development of the **QuickShare – Serverless Web Application** follows a **cloud-native** and **event-driven** architecture using Amazon Web Services (AWS). The methodology focuses on designing a scalable, secure, and efficient application that minimizes infrastructure management while providing seamless communication between the frontend, backend, and database. By adopting serverless computing, the application executes backend logic only when user requests are received, eliminating the need to manage traditional application servers.

The application consists of a **frontend**, a **serverless backend**, and a **MongoDB database**. The frontend is developed using **REACT, and JavaScript**, providing users with an interactive interface for creating and retrieving shared content. The backend is implemented using **AWS Lambda**, which executes the application's business logic in response to HTTP requests. Communication between the frontend and backend is handled through **Amazon API Gateway**, which exposes secure REST API endpoints and routes requests to the appropriate Lambda functions. **MongoDB** is used as the NoSQL database to store and retrieve application data efficiently.

The overall methodology consists of the following phases:

Phase 1: Application Development

The frontend of the application is developed using standard web technologies including **REACT, and JavaScript** to provide a responsive and user-friendly interface. The backend logic is implemented using **Node.js** in AWS Lambda, where each Lambda function is responsible for processing incoming requests, validating user input, and interacting with the MongoDB database. This modular approach improves code maintainability, scalability, and

application performance.

Phase 2: API Integration

Amazon API Gateway is configured to expose secure REST API endpoints that enable communication between the frontend and backend. Whenever a user performs an operation such as creating or retrieving shared content, the frontend sends an HTTP request to the corresponding API Gateway endpoint.

API Gateway receives the request, validates it, and securely invokes the appropriate AWS Lambda function. After processing the request, the Lambda function returns a JSON response through API Gateway back to the frontend.

Phase 3: Serverless Request Processing

AWS Lambda serves as the serverless compute layer of the application. When triggered by API Gateway, Lambda executes the required business logic, processes incoming request data, communicates with the MongoDB database, and generates the appropriate response.

Since Lambda follows an event-driven execution model, computing resources are allocated only during request execution, reducing infrastructure costs while providing automatic scalability and efficient resource utilization.

Phase 4: Database Management

The application uses **MongoDB** as the primary NoSQL database for storing and retrieving shared content. AWS Lambda establishes a secure connection with the MongoDB database whenever data needs to be inserted or retrieved.

MongoDB offers flexible document-based storage, allowing the application to efficiently manage user-generated data while supporting fast query performance and easy scalability.

Phase 5: Monitoring and Logging

Amazon CloudWatch continuously monitors the application by collecting execution logs, request metrics, and performance statistics from AWS Lambda and API Gateway. Developers can analyze these logs to monitor request execution, identify runtime issues, and troubleshoot application errors efficiently.

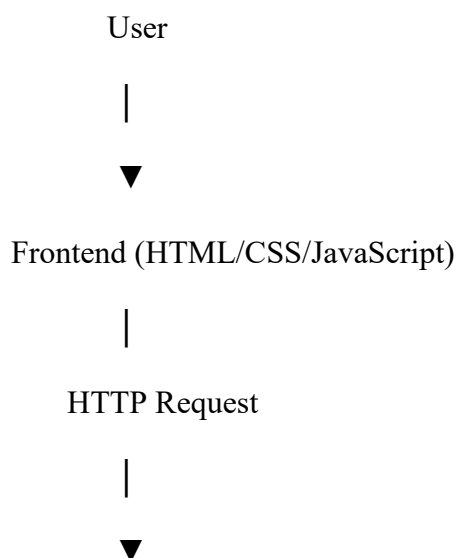
CloudWatch provides valuable insights into application health and helps maintain reliable system performance.

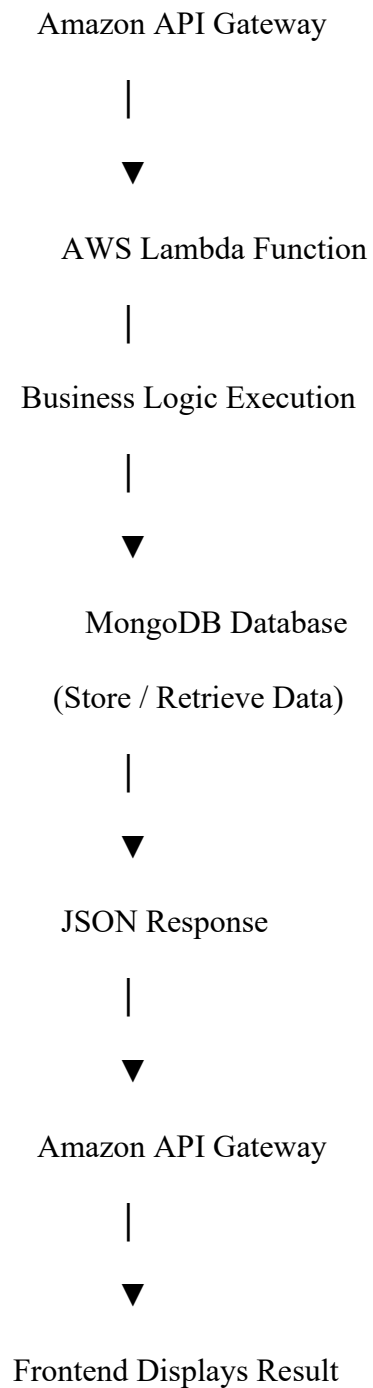
Phase 6: Security Management

AWS Identity and Access Management (IAM) is used to securely manage permissions between AWS services. IAM roles ensure that AWS Lambda functions have only the required permissions to access API Gateway, CloudWatch, and other AWS resources.

This implementation follows the **Principle of Least Privilege**, improving the overall security of the application.

Workflow:





3. SYSTEM DESIGN / ARCHITECTURE

3.1 System Overview

The **QuickShare** Serverless Web Application is designed using **Amazon Web Services**

(AWS) to provide a secure, scalable, and efficient platform for storing and retrieving shared information. The application follows a **serverless architecture**, where the backend business logic is executed on demand without requiring dedicated application servers. This architecture minimizes infrastructure management while providing automatic scalability, high availability, and cost-efficient resource utilization.

The application consists of a responsive frontend developed using **REACT**, and **JavaScript**, a serverless backend powered by **AWS Lambda**, and **Amazon API Gateway**, which serves as the communication layer between the client application and backend services. User data is stored in **MongoDB**, a document-oriented NoSQL database that provides flexible data storage and fast query performance. **AWS Identity and Access Management (IAM)** is used to control secure access between AWS services, while **Amazon CloudWatch** monitors application logs and execution metrics for performance analysis and troubleshooting.

The proposed architecture enables seamless communication between the frontend and backend while ensuring secure request processing and efficient data management. Since AWS automatically manages the serverless infrastructure, developers can focus on implementing application functionality instead of maintaining servers or operating systems.

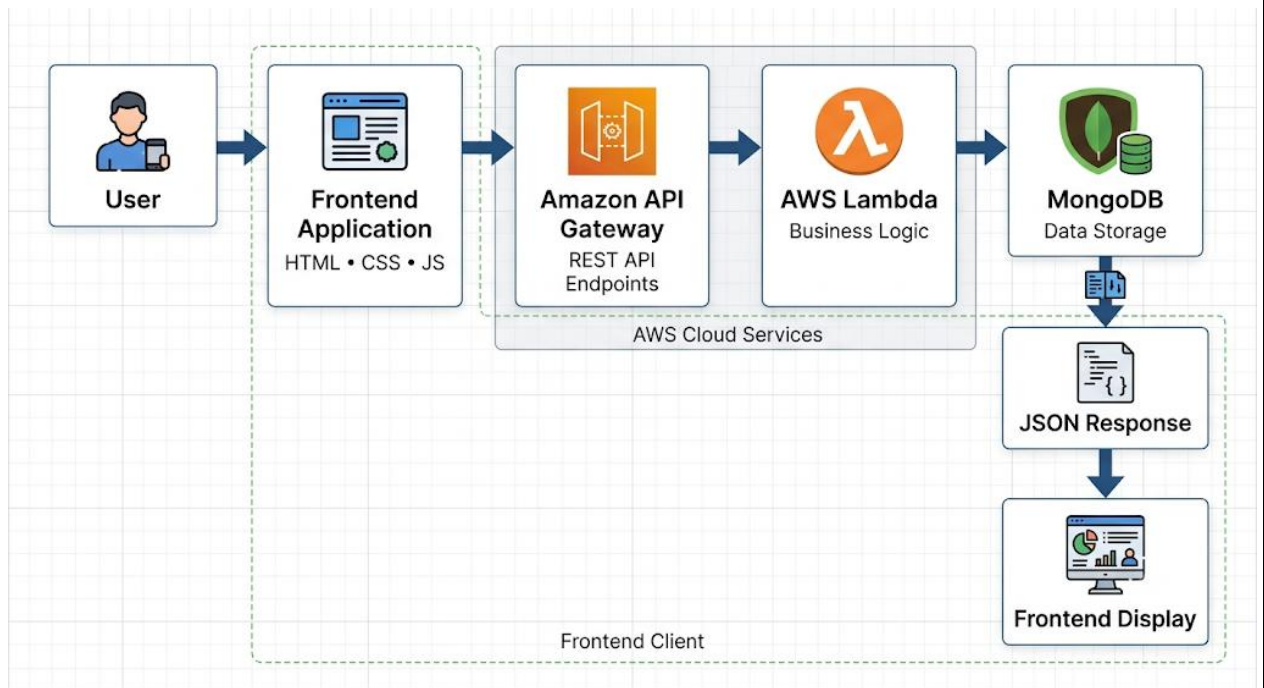
3.2 Overall System Architecture

Architecture Description

The architecture consists of the following components:

- The user accesses the QuickShare web application using a web browser.
- The frontend interface is developed using REACT, and JavaScript.
- User actions generate HTTP requests that are sent to Amazon API Gateway.
- API Gateway validates and forwards requests to the corresponding AWS Lambda function.

- AWS Lambda executes the business logic and processes user requests.
- Lambda securely communicates with the MongoDB database to store or retrieve application data.
- The processed data is returned as a JSON response through API Gateway.
- The frontend receives the response and dynamically displays the results to the user.



3.3 Frontend Architecture

The frontend of the QuickShare application is built using **REACT**, and **JavaScript**. It provides a clean and responsive user interface through which users can interact with the application. JavaScript is responsible for sending asynchronous HTTP requests to API Gateway using the Fetch API and displaying the returned responses dynamically without requiring a page refresh.

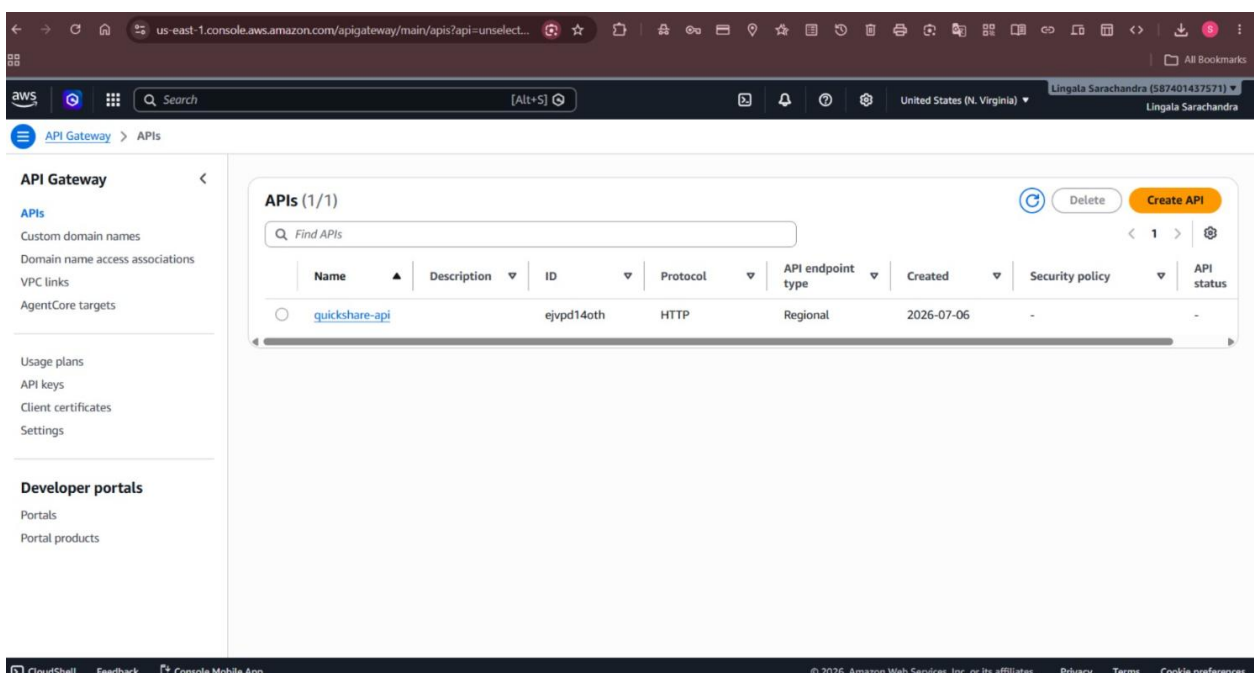
Figure 3.1 QuickShare Home Page

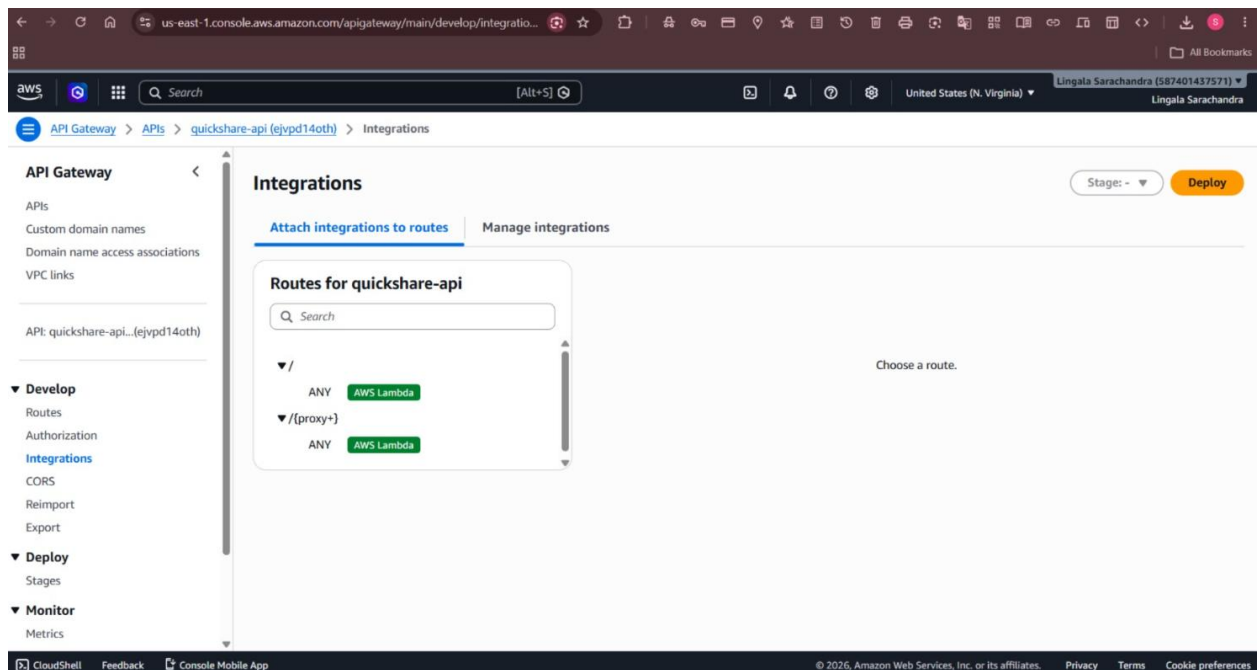


3.4 Amazon API Gateway

Amazon API Gateway acts as the secure entry point for all client requests. It exposes REST API endpoints that receive HTTP requests from the frontend application and routes them to the appropriate AWS Lambda functions. API Gateway also manages request validation, CORS configuration, and secure communication between the client and backend.

Figure 3.2 API Gateway Configuration

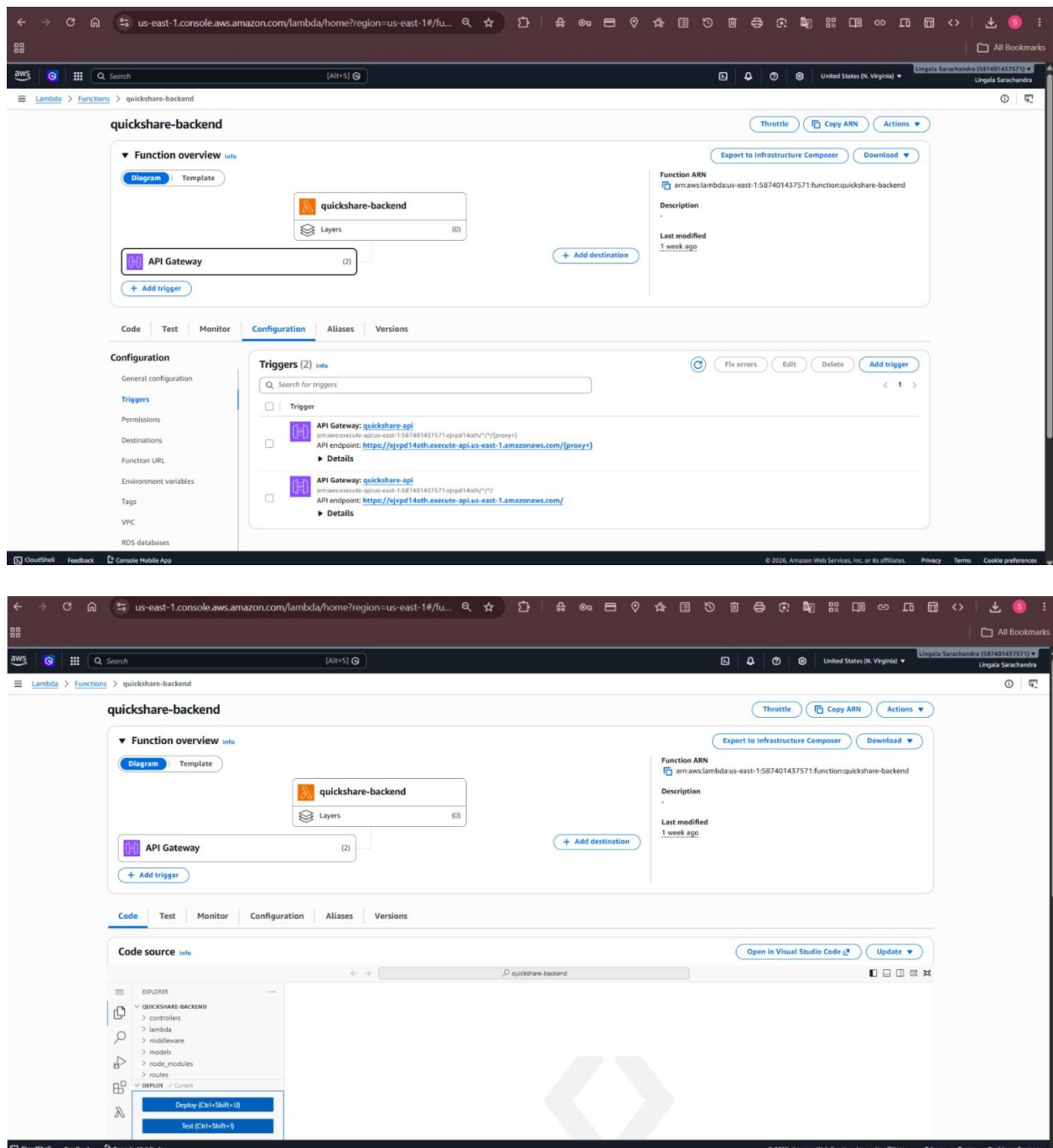




3.5 AWS Lambda

AWS Lambda provides the serverless compute environment for executing the backend business logic. Whenever an API request is received, Lambda processes the request, validates the input, performs the required operations, communicates with MongoDB, and generates a JSON response. Lambda automatically scales according to incoming traffic and executes only when invoked.

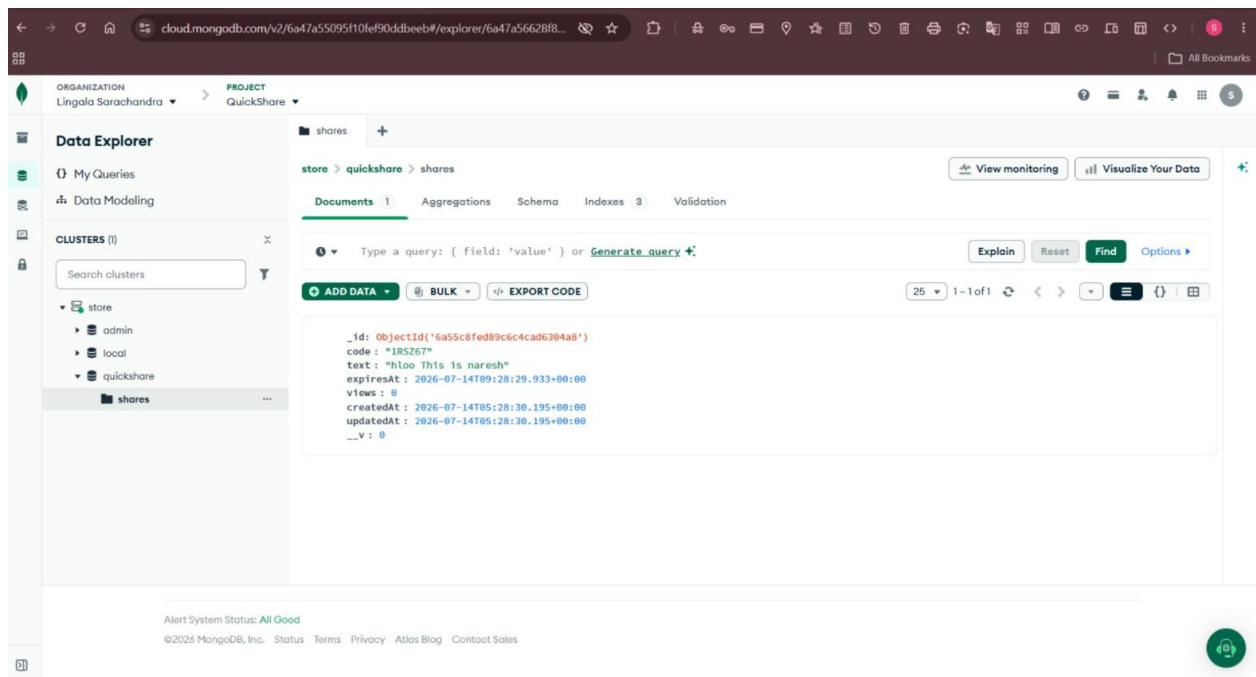
Figure 3.3 AWS Lambda Function



3.6 MongoDB Database

MongoDB is used as the application's NoSQL database to store and retrieve shared information. The document-oriented data model allows flexible storage of user data and provides fast query performance. AWS Lambda establishes a secure connection with MongoDB whenever database operations are required.

Figure 3.4 MongoDB Database Collections



3.7 AWS IAM and CloudWatch

AWS Identity and Access Management (IAM) is used to securely manage permissions for AWS services. IAM roles ensure that Lambda functions have only the necessary permissions required to execute application operations. Amazon CloudWatch continuously monitors Lambda execution logs, API Gateway requests, and performance metrics, enabling developers to identify errors and optimize application performance.

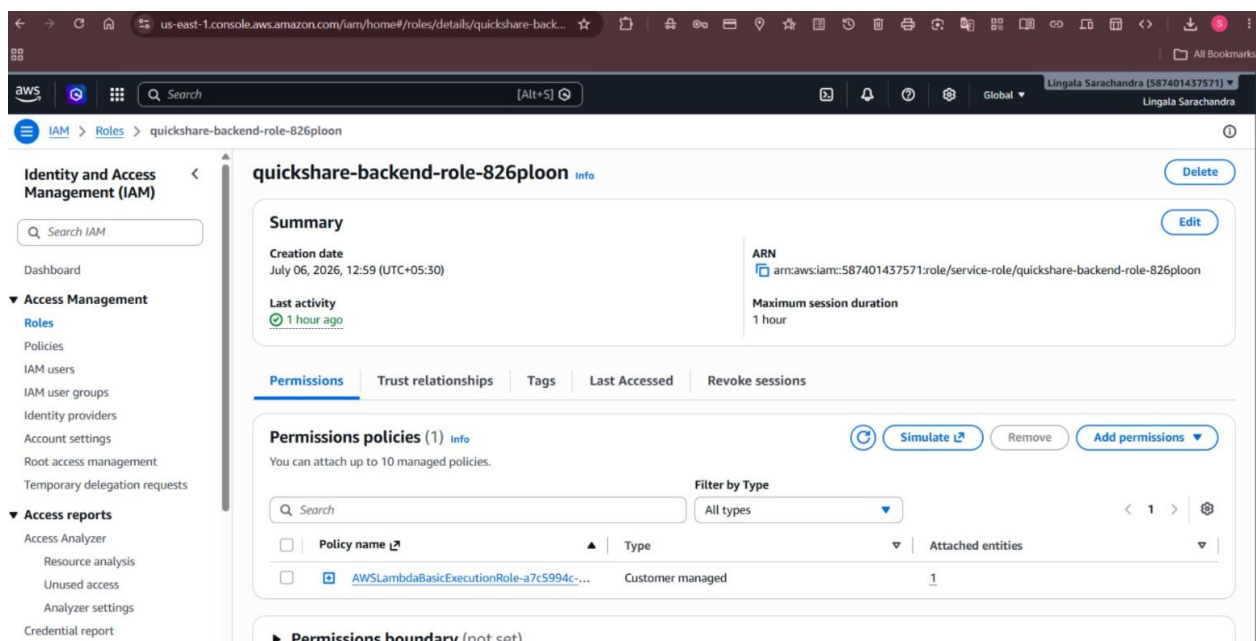
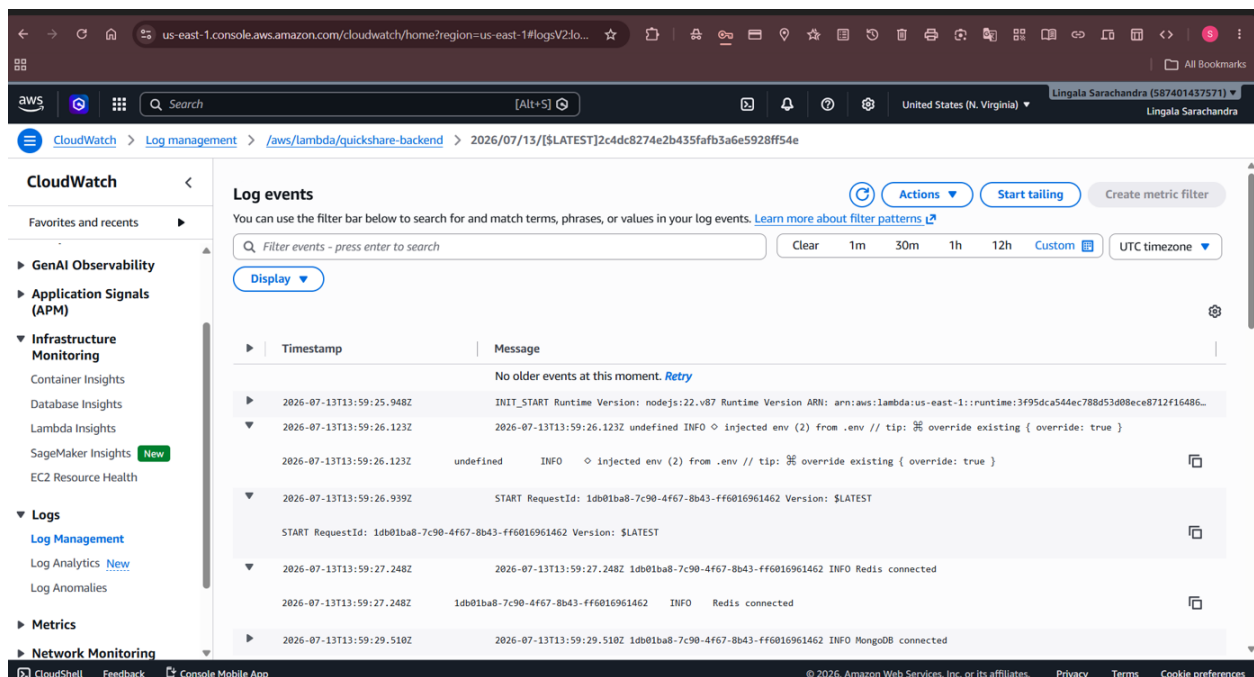


Figure 3.5 CloudWatch Monitoring Dashboard



3.8 Architecture Advantages

The proposed architecture provides several benefits:

- Fully serverless architecture with no server management.
- Automatic scaling based on incoming requests.
- Secure REST API communication using Amazon API Gateway.
- Efficient backend processing through AWS Lambda.
- Flexible NoSQL data storage using MongoDB.
- Role-based security using AWS IAM.
- Continuous monitoring using Amazon CloudWatch.
- Reduced infrastructure costs with a pay-as-you-use model.
- Modular and maintainable application design.

3.9 Architecture Summary

The QuickShare application successfully integrates **Amazon API Gateway**, **AWS Lambda**, **MongoDB**, **AWS IAM**, and **Amazon CloudWatch** to create a secure and scalable serverless web application. User requests are processed through an event-driven architecture, where AWS Lambda executes business logic and interacts with MongoDB for data storage and retrieval. The use of AWS managed services minimizes infrastructure management while providing high availability, automatic scaling, and reliable application performance, making the system suitable for modern cloud-native web applications.

4. IMPLEMENTATION

The implementation The implementation of the **QuickShare – Serverless Web Application** demonstrates the practical use of **Amazon Web Services (AWS)** to build a secure, scalable, and serverless cloud application. The application integrates multiple AWS managed services to process user requests, execute backend business logic, and store application data without requiring traditional web servers. The serverless architecture minimizes infrastructure management while ensuring high availability, automatic scaling, and efficient resource utilization.

The application consists of a responsive frontend developed using **REACT**, and **JavaScript**, a serverless backend implemented using **AWS Lambda**, **Amazon API Gateway** for secure REST API communication, and **MongoDB** for persistent data storage. **AWS Identity and Access Management (IAM)** provides secure access control, while **Amazon CloudWatch** monitors application performance and execution logs.

4.1 Frontend Development

The frontend of the QuickShare application was developed using **REACT**, and **JavaScript**. The user interface allows users to create and retrieve shared content through an interactive and responsive web page.

The frontend performs the following functions:

- Accepts user input.
- Sends HTTP requests using the Fetch API.
- Receives JSON responses from the backend.
- Displays results dynamically without refreshing the page.

Key Features

- Responsive user interface

- Client-side validation
- Asynchronous API communication
- Dynamic content updates

Insert Screenshot: QuickShare Home Page

4.2 Amazon API Gateway Configuration

Amazon API Gateway serves as the communication layer between the frontend and the backend. It exposes REST API endpoints that receive HTTP requests from users and securely invoke the corresponding AWS Lambda functions.

The API Gateway performs the following operations:

1. Receives HTTP requests.
2. Validates incoming requests.
3. Routes requests to AWS Lambda.
4. Returns JSON responses to the frontend.

Key Features

- REST API Endpoints
- HTTPS Communication
- Request Routing
- CORS Configuration
- Secure API Access

Insert Screenshot: API Gateway Resources

4.3 Backend Processing using AWS Lambda

The backend business logic is implemented using **AWS Lambda** with the **Node.js runtime**.

Lambda functions execute only when invoked by API Gateway, eliminating the need for

continuously running servers.

The Lambda function performs the following tasks:

1. Receives the API request.
2. Validates request parameters.
3. Connects to MongoDB.
4. Stores or retrieves application data.
5. Generates a JSON response.
6. Returns the response through API Gateway.

Key Lambda Details

- Runtime: Node.js 18.x
- Trigger: Amazon API Gateway
- Execution Model: Event-Driven
- Logging: Amazon CloudWatch
- Security: IAM Role

Insert Screenshot: AWS Lambda Function Dashboard

4.4 MongoDB Database Integration

MongoDB is used as the application's NoSQL database for storing and retrieving shared information. The database provides flexible document-based storage and supports efficient CRUD (Create, Read, Update, Delete) operations.

AWS Lambda establishes a secure connection with MongoDB to perform all database transactions.

Database Features

- NoSQL Document Database
- Flexible Schema

- High Performance
- Fast Query Execution
- Scalable Data Storage

Insert Screenshot: MongoDB Collections

4.5 IAM Security Configuration

AWS Identity and Access Management (IAM) is used to provide secure access between AWS services. IAM roles ensure that Lambda functions have only the permissions required to execute application operations.

The configured permissions include:

- Lambda execution permissions
- API Gateway invocation permissions
- CloudWatch logging permissions

This implementation follows the **Principle of Least Privilege**, improving the overall security of the application.

Insert Screenshot: IAM Role Configuration

4.6 Monitoring using Amazon CloudWatch

Amazon CloudWatch continuously monitors application performance and records execution logs generated by AWS Lambda and API Gateway.

CloudWatch provides detailed information about:

- Lambda execution logs
- API request logs
- Error messages
- Execution duration

- Invocation count

The monitoring service simplifies troubleshooting and improves application reliability.

CloudWatch Metrics

- Invocation Count
- Execution Duration
- Error Rate
- Request Logs

Insert Screenshot: CloudWatch Logs

4.7 Application Workflow

The QuickShare application follows the workflow shown below.

User

|



Frontend (HTML/CSS/JavaScript)

|

HTTP Request

|



Amazon API Gateway

|



AWS Lambda

|

Business Logic Execution

|



MongoDB

(Store / Retrieve Data)

|



JSON Response

|



Frontend Displays Result

4.8 Implementation Summary

The successful implementation of the **QuickShare** application demonstrates how AWS managed services can be integrated to build a secure, scalable, and efficient serverless web application. **Amazon API Gateway** securely handles client requests, **AWS Lambda** executes backend business logic, and **MongoDB** provides reliable NoSQL data storage. **AWS IAM** ensures secure access control, while **Amazon CloudWatch** continuously monitors application performance and execution logs.

The serverless architecture eliminates the need for managing physical servers, automatically scales according to user requests, and reduces operational costs. This implementation provides a strong foundation for future enhancements such as user authentication, file sharing, notification services, and additional serverless features, making QuickShare a modern cloud-native web application.

5. RESULTS

The implementation of the **QuickShare – Serverless Web Application** was successfully completed using **Amazon Web Services (AWS)**. The application was tested to verify the functionality of each component, including the frontend interface, Amazon API Gateway, AWS Lambda, MongoDB database integration, IAM security configuration, and CloudWatch monitoring. The testing confirmed that the application performs reliably and efficiently under normal operating conditions.

The frontend application successfully communicated with the backend through **Amazon API Gateway** using REST API requests. Every request generated from the web interface was correctly routed to the corresponding **AWS Lambda** function, where the business logic was executed. The Lambda function processed user requests, interacted with the **MongoDB** database for storing and retrieving information, and returned structured JSON responses to the frontend without any communication errors.

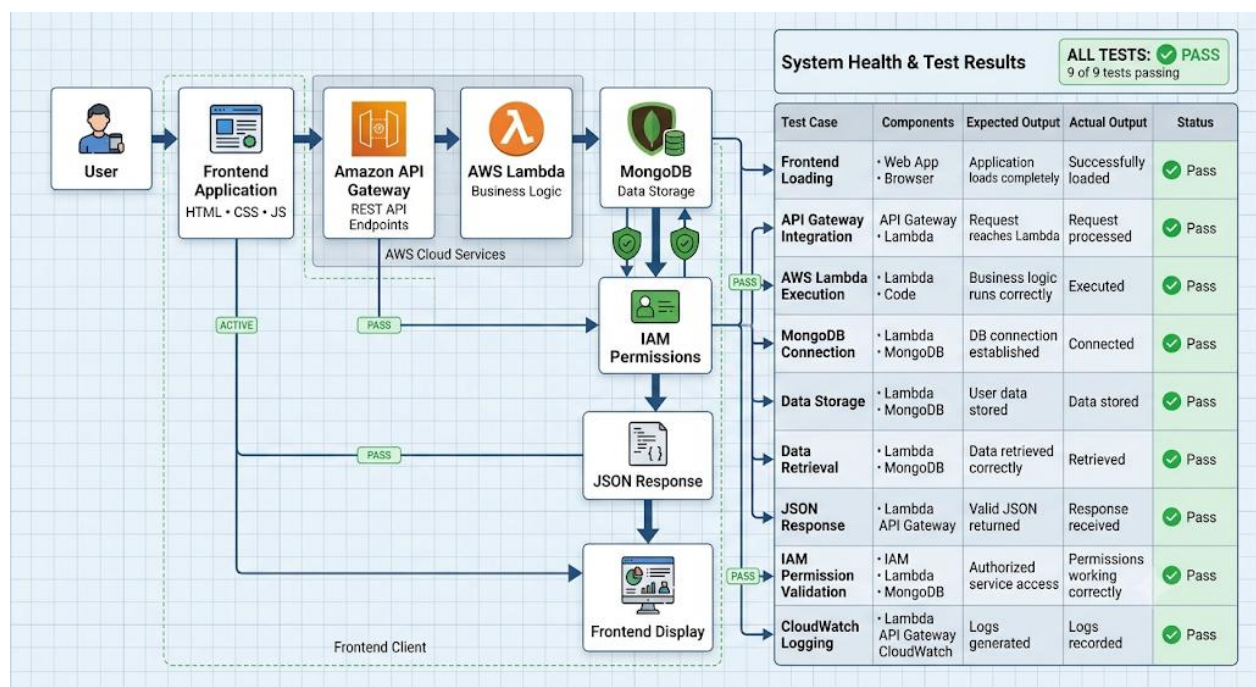
The integration between AWS Lambda and MongoDB was successfully validated. The application was able to establish a secure database connection, perform data insertion and retrieval operations, and return accurate results to users. The use of MongoDB provided flexible document-based storage with fast query execution, contributing to the overall performance and responsiveness of the application.

Amazon CloudWatch successfully recorded Lambda execution logs, API requests, and runtime information throughout the testing process. These logs enabled effective monitoring of application performance and simplified debugging by providing detailed information about request execution and potential errors. Additionally, **AWS Identity and Access Management (IAM)** roles functioned correctly by granting only the required permissions to AWS resources, ensuring secure communication between the services.

The serverless architecture demonstrated several advantages during implementation,

including automatic request handling, reduced infrastructure management, and efficient resource utilization. Since AWS Lambda executes code only when requests are received, the application consumed computing resources only during execution, making the solution both scalable and cost-effective.

Overall, the QuickShare application achieved its intended objectives by providing a secure, reliable, and serverless platform for processing user requests and managing application data. The successful integration of AWS managed services confirmed the effectiveness of the proposed architecture and demonstrated the practical implementation of serverless computing for modern cloud-based web applications.



Result Analysis

The experimental results demonstrate that the **QuickShare Serverless Web Application** performs efficiently using AWS serverless services. The application successfully handled client requests through **Amazon API Gateway**, executed backend logic using **AWS Lambda**, and managed data storage and retrieval through **MongoDB**. The monitoring

capabilities of **Amazon CloudWatch** and the secure access control provided by **AWS IAM** ensured reliable system performance and secure communication between AWS resources.

The successful execution of all functional test cases confirms that the proposed architecture is suitable for developing modern cloud-native applications. The serverless approach reduced infrastructure management while providing automatic scalability, high availability, and efficient resource utilization. The project serves as a strong foundation for future enhancements such as user authentication, file sharing, notification services, and integration with additional AWS services.

6. CONCLUSION

The **QuickShare – Serverless Web Application** successfully demonstrates the implementation of a modern cloud-native application using **Amazon Web Services (AWS)**. By integrating **Amazon API Gateway**, **AWS Lambda**, **MongoDB**, **AWS Identity and Access Management (IAM)**, and **Amazon CloudWatch**, the project provides a secure, scalable, and efficient platform for processing user requests without the need to manage traditional server infrastructure. The serverless architecture enables the application to execute backend functions on demand, reducing operational complexity and improving resource utilization.

Throughout the implementation, the application successfully handled user requests, executed backend business logic, and performed data storage and retrieval operations using MongoDB. The integration between the frontend and backend through Amazon API Gateway ensured seamless communication, while AWS Lambda efficiently processed requests and returned accurate responses. CloudWatch monitoring and IAM-based access control further enhanced the reliability, security, and maintainability of the application.

One of the major advantages of the proposed solution is its ability to automatically scale according to user demand while following a **pay-as-you-use** pricing model. Since AWS manages the underlying infrastructure, developers can focus on application development instead of server administration, software updates, and infrastructure maintenance. This significantly reduces deployment complexity and operational costs while improving application availability and performance.

The project also demonstrates the practical benefits of serverless computing in developing lightweight, modular, and highly available web applications. The event-driven architecture allows each component to operate independently, making the system easier to maintain, extend, and integrate with additional cloud services in the future.

In conclusion, the **QuickShare** project successfully achieves its objectives by delivering a secure, reliable, and scalable serverless web application using AWS managed services. It serves as a practical example of cloud-native application development and provides a strong foundation for future enhancements such as user authentication, file upload and sharing, custom domain integration, content delivery using **Amazon CloudFront**, advanced monitoring, and the addition of new serverless features. The knowledge and experience gained through this project provide valuable insights into modern cloud computing technologies and their real-world applications in developing enterprise-ready web solutions.
